

02 — B-Tree Indexes

"The B-tree is the workhorse of database indexing — balanced, ordered, and universally applicable."

Table of Contents

1. [Learning Objectives](#)
 2. [B-Tree Data Structure](#)
 3. [PostgreSQL B-Tree Page Layout](#)
 4. [ASCII Diagram: Full B-Tree Structure](#)
 5. [Equality Searches](#)
 6. [Range Scans](#)
 7. [Backward Scans](#)
 8. [Index Creation and Options](#)
 9. [How B-Tree Handles NULL](#)
 10. [Multi-Column B-Tree Indexes](#)
 11. [EXPLAIN Output for B-Tree Scans](#)
 12. [Deduplication in PostgreSQL 13+](#)
 13. [Common Mistakes](#)
 14. [Best Practices](#)
 15. [Performance Considerations](#)
 16. [Interview Questions & Answers](#)
 17. [Exercises with Solutions](#)
 18. [Production Scenarios](#)
 19. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Describe the B-tree structure: root, internal pages, leaf pages, and sibling links
 - Trace the path of an equality lookup and a range scan through a B-tree
 - Explain page splits and how they maintain balance
 - Read EXPLAIN output showing Index Scan, Index Only Scan, and Bitmap Index Scan
 - Configure B-tree indexes with fill factor, deduplication, and index options
 - Identify when a B-tree index cannot be used despite being present
-

B-Tree Data Structure

A **B-tree** (Balanced Tree) is a self-balancing search tree where:

- Every leaf node is at the same depth (perfectly balanced)
- Each internal node contains **k** keys and **k+1** child pointers
- Keys within each node are stored in **sorted order**
- Leaf nodes contain the actual index entries (key + ctid)
- Leaf nodes are linked in a **doubly-linked list** for efficient range scans

The "B" in B-tree stands for balanced (sometimes attributed to Bayer, one of its inventors).

Key Properties

Property	Value for PostgreSQL
Page size	8 KB
Keys per internal page	~400 (for 4-byte integers)
Keys per leaf page	~300 (for 4-byte integers + ctid)
Tree height for 100M rows	~4 levels
Equality lookup cost	~4 page reads (root + 2 internal + 1 leaf)

Because internal pages have hundreds of entries, B-trees are very **shallow** — almost all practical databases fit in 3–5 levels regardless of table size.

PostgreSQL B-Tree Page Layout

PostgreSQL B-tree pages (nbtree pages) have this internal layout:

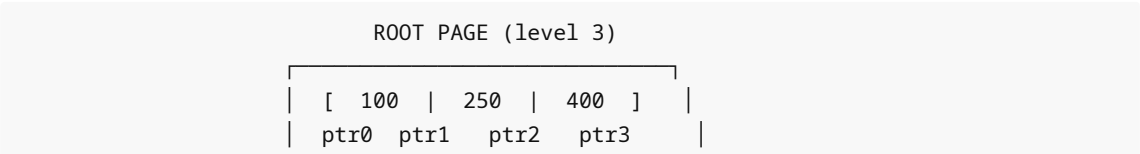
```
+-----+
| Page Header (24 bytes) |
| lsn, checksum, flags, free space info |
+-----+
| Opaque Data (16 bytes) |
| btpo_prev, btpo_next (sibling links) |
| btpo_level (0 = leaf) |
| btpo_flags (BTP_LEAF, BTP_ROOT, etc.) |
+-----+
| Item ID Array (4 bytes each) |
| [offset_to_item_1 | item_1_flags] |
| [offset_to_item_2 | item_2_flags] |
| ... |
+-----+
| Free Space |
+-----+
| Items (grow upward from end of page) |
| item_N: [key_value | ctid | heap_tid] |
| item_2: [key_value | ctid | heap_tid] |
| item_1: [key_value | ctid | heap_tid] |
+-----+
```

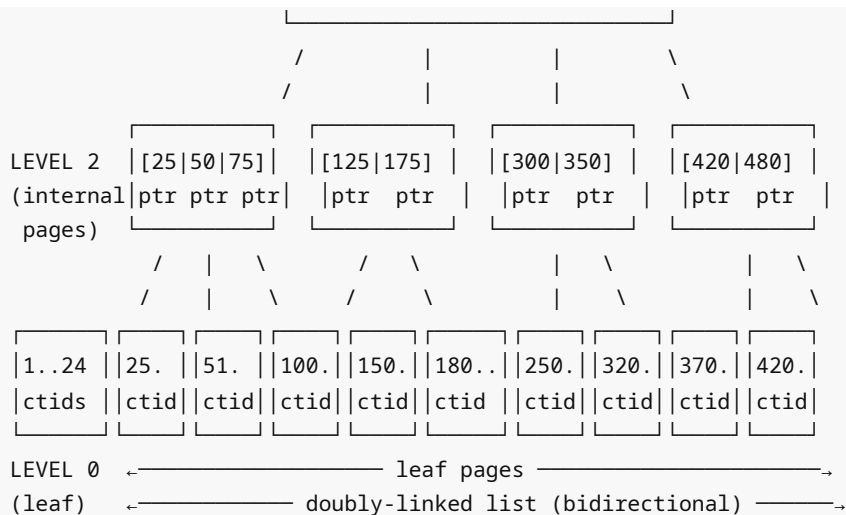
Internal page items contain: key_value + child_block_number

Leaf page items contain: key_value + heap_tid (ctid pointing to heap row)

ASCII Diagram: Full B-Tree Structure

This diagram shows a B-tree index on `orders(customer_id)` with ~200 rows:





Leaf page detail:

prev_page_ptr	PAGE HEADER	next_page_ptr
key=1 ctid=(0,1)	key=2 ctid=(0,2)	
key=3 ctid=(0,3)	key=4 ctid=(1,1)	...
key=22 ctid=(5,4)	key=24 ctid=(5,6)	

ctid = (page_number, tuple_offset_on_page)
e.g. ctid=(5,4) means: heap page 5, 4th tuple on that page

Tree Height vs Table Size

Rows	Tree Height	Lookups needed
< 100	1 (root=leaf)	1
< 500	2	2
< 250,000	3	3
< 125,000,000	4	4
< 60 billion	5	5

Equality Searches

For the query `WHERE customer_id = 175` :

Step 1: Read root page
Root: [100 | 250 | 400]
175 > 100, 175 < 250 → follow ptr1

Step 2: Read internal page (ptr1 from root)
Page: [125 | 175]
175 >= 175 → follow ptr for value >= 175

Step 3: Read leaf page
Scan leaf: find key=175, extract ctids
Found: ctid=(3,7), ctid=(3,8)

Step 4: Fetch heap pages
Read heap page 3, return tuples 7 and 8

Total I/Os: 4 (regardless of table size, given height = 3)

Range Scans

For the query `WHERE customer_id BETWEEN 150 AND 200` :

Step 1: Navigate tree to find first key ≥ 150
(same traversal as equality, lands on leaf containing key=150)

Step 2: Scan forward through leaf pages using next_page pointers
Leaf page 1: keys 150, 155, 160, 165, 170, 175 → collect ctids
Follow next_page pointer →
Leaf page 2: keys 180, 185, 190, 195, 200 → collect ctids
key 205 > 200, STOP

Step 3: Fetch matching heap pages (in ctid order if Bitmap scan, random order if Index Scan)

This is why sorted leaf pages with sibling links make B-trees excellent for range queries.

Backward Scans

PostgreSQL B-trees support **backward scans** natively. For `ORDER BY customer_id DESC` :

Step 1: Navigate tree to find last key (right-most leaf)
Step 2: Scan backward through leaf pages using prev_page pointers
Step 3: Return tuples in descending order

This means `CREATE INDEX ON orders(customer_id)` supports both:

- `ORDER BY customer_id ASC`
- `ORDER BY customer_id DESC`

without a separate index, because the index can be traversed in both directions.

For multi-column, `ORDER BY customer_id ASC, created_at DESC` requires an index with matching sort directions: `CREATE INDEX ON orders(customer_id ASC, created_at DESC)` .

Index Creation and Options

Sort Order

```
-- Ascending (default)
CREATE INDEX ON orders(created_at ASC);

-- Descending
CREATE INDEX ON orders(created_at DESC);

-- NULLs first (default for DESC)
CREATE INDEX ON orders(created_at DESC NULLS LAST);

-- Multi-column with mixed sort
CREATE INDEX ON orders(customer_id ASC, created_at DESC NULLS LAST);
```

Fill Factor

The fill factor controls how full each page is when initially built. Lower fill factor leaves space for future updates, reducing page splits on hot pages.

```
-- Default fill factor = 90 for leaf pages
CREATE INDEX ON orders(customer_id) WITH (fillfactor = 70);
-- Useful for tables with heavy UPDATE workloads on indexed columns
```

Deduplication (PostgreSQL 13+)

```
-- Deduplication enabled by default in PG13+
-- Disable if keys are mostly unique and you need to save CPU
CREATE INDEX ON orders(status) WITH (deduplicate_items = off);
```

Fast Build (maintenance_work_mem)

```
-- Increase memory for faster index builds (session level)
SET maintenance_work_mem = '2GB';
CREATE INDEX ON orders(customer_id);
-- More memory = fewer sort passes = faster build
```

How B-Tree Handles NULL

PostgreSQL B-tree indexes **do** index NULL values.

- NULL values are stored at one end of the sorted order
- `NULLS FIRST` (default for DESC): NULLs stored at the left/low end
- `NULLS LAST` (default for ASC): NULLs stored at the right/high end
- Queries like `WHERE column IS NULL` **can** use a B-tree index

```
-- Both these queries can use a B-tree index on email
WHERE email IS NULL
```

```
WHERE email IS NOT NULL
WHERE email = 'foo@bar.com'
```

Important: A B-tree index on a column does not prevent NULL values unless you add a NOT NULL constraint or a partial index `WHERE column IS NOT NULL`.

Multi-Column B-Tree Indexes

A composite B-tree index sorts by (col1, col2, ..., colN) lexicographically.

```
CREATE INDEX ON orders(customer_id, created_at, status);
```

Prefix Matching Rule

The index can be used for queries that filter on a **prefix** of the index columns:

Query Predicate	Uses Index?	Notes
WHERE customer_id = 1	Yes	Uses first column
WHERE customer_id = 1 AND created_at > '2024-01-01'	Yes	Uses both columns
WHERE customer_id = 1 AND status = 'pending'	Partial	customer_id used, status skipped
WHERE created_at > '2024-01-01'	No	Missing leading column
WHERE status = 'pending'	No	Missing leading columns

Index Visual for Composite Key

```
Index on (customer_id, created_at):

Leaf pages sorted as:
[1, 2024-01-01] [1, 2024-01-05] [1, 2024-03-12]
[2, 2024-02-01] [2, 2024-02-15]
[3, 2024-01-01] [3, 2024-05-20]

Query: WHERE customer_id = 2 AND created_at > '2024-02-10'
→ Navigate to first entry with customer_id=2 and created_at>2024-02-10
→ Scan forward: [2, 2024-02-15] matches; [3, ...] stop (customer_id changed)
→ Efficient!

Query: WHERE created_at > '2024-02-10'
→ Cannot use index – no way to skip to matching created_at without scanning ALL customer_ids
→ Full index scan (as bad as Seq Scan)
```

EXPLAIN Output for B-Tree Scans

Index Scan

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 42;
```

```
Index Scan using idx_orders_customer on orders  (cost=0.56..8.58 rows=5 width=48)
                                                    (actual time=0.023..0.031 rows=5
loops=1)
  Index Cond: (customer_id = 42)
  Buffers: shared hit=4
Planning Time: 0.123 ms
Execution Time: 0.051 ms
```

- cost=0.56..8.58 — startup cost .. total cost
- rows=5 — planner estimated 5 rows
- actual rows=5 — actual rows returned
- shared hit=4 — 4 pages found in shared_buffers (no disk I/O)

Index Only Scan (covering index)

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT customer_id, total_amount FROM orders WHERE customer_id = 42;
-- Index includes customer_id (key) + total_amount (INCLUDE clause)
```

```
Index Only Scan using idx_orders_cust_inc_total on orders
  (cost=0.56..4.58 rows=5 width=12)
  (actual time=0.018..0.022 rows=5 loops=1)
  Index Cond: (customer_id = 42)
  Heap Fetches: 0          ← no heap access needed!
  Buffers: shared hit=3
```

Bitmap Index Scan + Bitmap Heap Scan

Used when many rows match (too many for random index scan, too few for seq scan):

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id BETWEEN 100 AND 200;
```

```
Bitmap Heap Scan on orders  (cost=23.45..856.12 rows=500 width=48)
                                                    (actual time=0.234..2.456 rows=512 loops=1)
  Recheck Cond: ((customer_id >= 100) AND (customer_id <= 200))
  Heap Blocks: exact=489
  Buffers: shared hit=512
  -> Bitmap Index Scan on idx_orders_customer
        (cost=0.00..23.33 rows=500 width=0)
```

```
(actual time=0.189..0.189 rows=512 loops=1)
Index Cond: ((customer_id >= 100) AND (customer_id <= 200))
```

The bitmap scan collects all ctids first, sorts them by heap page, then reads heap pages in sequential order — more efficient than random reads when many rows match.

Deduplication in PostgreSQL 13+

When multiple heap rows have the same index key value (e.g., many orders for customer 42), PostgreSQL 13+ can **deduplicate** leaf page entries:

```
Before deduplication:
[42, ctid=(1,1)] [42, ctid=(1,2)] [42, ctid=(1,3)] [42, ctid=(2,1)]

After deduplication:
[42, {ctid=(1,1), ctid=(1,2), ctid=(1,3), ctid=(2,1)}]
```

This reduces leaf page size, meaning more entries fit per page, reducing index size by up to 90% for low-cardinality columns.

```
-- Check if deduplication is active
SELECT amname, pg_indexam_has_rule(oid, 'DEDUPLICATE')
FROM pg_am WHERE amname = 'btree';
```

Common Mistakes

1. Expecting left-to-right column usage to be automatic

The planner will NOT use an index on (a, b, c) for a query filtering only on b or c .

2. Using != or NOT IN and expecting index usage

```
-- BAD: does not use index efficiently
WHERE status != 'completed'
-- Planner usually prefers Seq Scan (too many rows returned)

-- BETTER: invert the logic if possible
WHERE status IN ('pending', 'processing')
```

3. Ignoring sort direction on multi-column indexes

```
-- Index on (customer_id ASC, created_at ASC)
-- This query uses the index:
ORDER BY customer_id ASC, created_at ASC
-- This query does NOT use the index directly for sort (needs re-sort):
ORDER BY customer_id ASC, created_at DESC
-- Fix:
CREATE INDEX ON orders(customer_id ASC, created_at DESC);
```

4. High fill factor on frequently updated tables

Using `fillfactor = 100` on a table with heavy UPDATES to indexed columns causes frequent page splits, leading to index bloat and slower performance.

5. Not rebuilding indexes after bulk loads

After a `COPY` or large batch INSERT, the index may have suboptimal page packing. `REINDEX CONCURRENTLY` can compact and re-sort the index.

Best Practices

1. **Default B-tree for most use cases** — unless you have a specific need for GIN, GiST, or BRIN, start with B-tree.
 2. **Set fill factor to 70-80** for tables with frequent UPDATES on indexed columns to reduce page splits and HOT update invalidation.
 3. **Use ASC/DESC and NULLS FIRST/LAST** to match your most common `ORDER BY` clauses — this enables index-only sort operations.
 4. **Keep index columns as narrow as possible** — a 4-byte integer index is ~3x smaller than a 16-byte UUID index, fitting more entries per page.
 5. **For UUID primary keys**, consider `gen_random_uuid()` fragmentation issues — UUIDs are random and cause index pages to split randomly. Consider `ulid` or sequential UUIDs (`uuid_generate_v1mc()`) to reduce fragmentation.
-

Performance Considerations

B-Tree Index Size Estimation

```
-- Rough size formula:
-- index_size ≈ rows × (key_size + ctid_size + overhead) / fill_factor / 8192
-- For orders(customer_id) with 10M rows, 4-byte int:
-- 10,000,000 × (4 + 6 + 8) / 0.9 / 8192 ≈ 23,000 pages ≈ 185 MB

-- Actual size:
SELECT pg_size_pretty(pg_relation_size('idx_orders_customer'));
```

Page Split Performance

When a leaf page is full and a new entry must be inserted:

1. PostgreSQL allocates a new page
2. Half the entries move to the new page
3. A new key is added to the parent internal page
4. If parent is also full, cascade split upward (rare)

Splits are expensive — minimize them with proper fill factor settings.

HOT Updates and Indexes

Heap Only Tuple (HOT) update: when an UPDATE changes only non-indexed columns, PostgreSQL can update the heap without touching the index. This is much faster.

```
-- Table: orders(id, customer_id, status, notes)
-- Index: on customer_id only

-- HOT update possible (status is not indexed):
UPDATE orders SET status = 'completed' WHERE id = 1;

-- HOT update NOT possible (customer_id is indexed):
UPDATE orders SET customer_id = 99 WHERE id = 1;
-- Must update both heap and index
```

Keep this in mind when designing indexes — fewer indexes on frequently-updated columns enables more HOT updates.

Interview Questions & Answers

Q1. What does "balanced" mean in B-tree and why does it matter?

A: Balanced means all leaf nodes are at the same depth. This guarantees that every lookup takes the same number of page reads regardless of which key you search for. Without balance, a tree could degrade to a linked list with $O(n)$ lookup time. PostgreSQL B-trees maintain balance automatically through page splits: when a page fills up, it splits into two pages and promotes a key to the parent, ensuring consistent depth.

Q2. Describe the path of an equality lookup in a B-tree index.

A: (1) Read the root page and compare the search key against the separator keys to find the correct child pointer. (2) Follow the pointer to the next internal page and repeat. (3) Continue until a leaf page is reached. (4) Scan the leaf page (which is sorted) to find entries matching the key. (5) Collect the ctids (heap row pointers) from those entries. (6) Fetch the corresponding heap pages. Total I/Os = tree height (typically 3–5).

Q3. How do B-tree indexes support range queries efficiently?

A: Leaf pages in a PostgreSQL B-tree are connected in a doubly-linked list sorted by key value. For a range query, the planner traverses the tree to find the first matching key, then follows the `next_page` pointer to scan forward through consecutive leaf pages until the range boundary is exceeded. This allows range scans to read only the relevant portion of the index in sequential (sorted) order without re-traversing the tree.

Q4. What is a page split and when does it occur?

A: A page split occurs when a leaf page is full and a new entry must be inserted. PostgreSQL allocates a new page, moves approximately half of the existing entries to the new page, updates the sibling links, and adds a separator key to the parent internal page. If the parent is also full, it splits recursively. Splits are expensive because they require exclusive locks and multiple page writes. They are minimized by setting a lower fill factor (e.g., 70-80%) which reserves free space in each page for future inserts.

Q5. What is HOT update and how does it relate to index design?

A: HOT (Heap Only Tuple) is an optimization where an UPDATE that does not change any indexed column can update the heap without writing to any index. The old tuple points to the new tuple in the heap chain, and the index still points to the old tuple which redirects to the new one. This avoids index write amplification. The implication for index design is that you should avoid indexing columns that are frequently updated, because each such update invalidates the HOT chain and requires writing to both heap and index.

Q6. Why does a composite index on (a, b) not help a query that only filters on b?

A: The B-tree sorts entries by (a, b) together — first by a, then by b within each a value. A query filtering only on b would need to find all values of b=X across all possible values of a. Since b values are interleaved among a values, the planner would have to scan the entire index — no better than a sequential scan. The index structure only allows efficient prefix-based lookups: queries must filter on the leftmost column(s) of the index definition.

Q7. How does PostgreSQL B-tree handle the LIKE operator?

A: B-tree supports `LIKE 'prefix%'` (prefix searches) because these are equivalent to a range scan: `WHERE name LIKE 'John%'` is equivalent to `WHERE name >= 'John' AND name < 'Joho'` (incrementing the last character). However, `LIKE '%suffix'` and `LIKE '%middle%'` cannot use a B-tree index because they don't correspond to a contiguous range of sorted keys. For those patterns use full-text search (GIN on tsvector) or trigram indexes (pg_trgm extension with GIN or GiST).

Q8. What is index deduplication (PostgreSQL 13+) and how does it affect performance?

A: Deduplication in PG13+ combines multiple leaf index entries that share the same key value into a single posting list entry. For example, if 100 orders have `customer_id=42`, instead of 100 separate leaf entries, there is one entry with a list of 100 ctids. This can reduce index size by 60–90% for low-cardinality columns, improving cache utilization and scan performance. The tradeoff is slightly more CPU work to encode/decode posting lists. It is enabled by default but can be disabled per-index with `deduplicate_items=off`.

Q9. When would you choose a DESC index over the default ASC index?

A: When the most common access pattern fetches the most recent or highest-value records. For example, `ORDER BY created_at DESC LIMIT 10` (latest records) benefits from a DESC index because the planner can read the first 10 leaf entries from the right side of the index without sorting. Without a DESC index, the planner reads the ASC index backward (also supported), but for multi-column indexes with mixed sort directions, a matching index is needed: `CREATE INDEX ON events(user_id ASC, created_at DESC)`.

Q10. What happens to a B-tree index during a VACUUM?

A: VACUUM removes dead index entries (from UPDATES and DELETES) from leaf pages. It marks pages that are completely empty as "deleted" (available for reuse). It does NOT shrink the index file or restructure the tree. Over time, vacuumed pages become available for new index entries without growing the file. REINDEX completely rebuilds the index from scratch, producing a compact, defragmented structure.

Exercises with Solutions

Exercise 1

You have `orders(id, customer_id, status, created_at, total)`. Design the best index for:

```
SELECT id, total FROM orders
WHERE customer_id = 5
      AND status = 'pending'
ORDER BY created_at DESC
LIMIT 10;
```

Solution:

```
-- Covering index: (customer_id, status, created_at DESC) INCLUDE (id, total)
-- Reasoning:
-- 1. customer_id first (equality, high selectivity)
-- 2. status second (equality)
-- 3. created_at DESC third (matches ORDER BY, allows index scan to stop after 10)
-- 4. INCLUDE id, total to enable index-only scan (avoid heap fetches)

CREATE INDEX CONCURRENTLY idx_orders_cust_status_date
ON orders(customer_id, status, created_at DESC)
INCLUDE (id, total);

-- Verify:
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, total FROM orders
WHERE customer_id = 5 AND status = 'pending'
ORDER BY created_at DESC LIMIT 10;
-- Expected: Index Only Scan
```

Exercise 2

Explain why the following index is NOT used for the given query:

```
CREATE INDEX idx_users_email ON users(email);
SELECT * FROM users WHERE UPPER(email) = 'ALICE@EXAMPLE.COM';
```

Solution: The index is on `email` but the query applies `UPPER()` function to it. The planner cannot use an index on `email` to satisfy `UPPER(email) = ...` because the index stores raw email values, not uppercased values. Fix:

```
CREATE INDEX idx_users_upper_email ON users(UPPER(email));
SELECT * FROM users WHERE UPPER(email) = 'ALICE@EXAMPLE.COM';
```

Production Scenarios

Scenario 1: UUID Primary Key Fragmentation

A SaaS application using `gen_random_uuid()` as primary key experiences severe index bloat and slow INSERT performance as the table grows past 50M rows.

Root cause: Random UUIDs insert into random leaf pages, causing:

- Constant page splits
- Most pages operating at ~50% fill after many splits
- Index 2x larger than necessary

Solution:

```
-- Option 1: Use UUIDv7 (time-ordered, available via extension or PG17+)
-- Option 2: Use BIGSERIAL instead of UUID for internal PK
-- Option 3: Use uuid_generate_v1mc() for time-based UUIDs

-- Check current fragmentation
CREATE EXTENSION IF NOT EXISTS pgstattuple;
SELECT * FROM pgstattuple('your_table_pkey');
-- Look at: avg_leaf_density (should be > 70%)

-- Rebuild the index
REINDEX CONCURRENTLY INDEX your_table_pkey;
```

Scenario 2: Slow ORDER BY with LIMIT on Large Table

```
-- Query: latest 10 orders for customer 42
-- Running in 8 seconds (full index scan + sort)
SELECT * FROM orders WHERE customer_id = 42 ORDER BY created_at DESC LIMIT 10;

-- Current index: orders(customer_id)
-- Problem: index finds all rows for customer 42, then sorts, then limits

-- Solution: composite index matching the full query pattern
CREATE INDEX CONCURRENTLY idx_orders_cust_date
ON orders(customer_id, created_at DESC);

-- Result: 0.1ms – planner reads exactly 10 entries from index leaf pages
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 42 ORDER BY created_at DESC LIMIT 10;
-- Index Scan using idx_orders_cust_date on orders
--   Index Cond: (customer_id = 42)
--   Rows Removed by Filter: 0
--   actual rows=10 loops=1
```

Cross-References

- [01_index_fundamentals.md](#) — Index basics and planner
- [09_composite_indexes.md](#) — Multi-column index design
- [10_covering_indexes.md](#) — INCLUDE clause and index-only scans
- [11_expression_indexes.md](#) — Functional indexes
- [12_index_maintenance.md](#) — REINDEX, bloat, VACUUM
- [../08_Query_Optimization/03_scan_types.md](#) — Index Scan vs Bitmap Scan